NAME: VAISHNAVI E ROLL NO: 231901059

EX. NO.9

MONITORING AND LOGGING FABRIC NETWORKS

AIM

To monitor the Hyperledger Fabric network by inspecting Docker containers, viewing peer and orderer logs, checking network status, monitoring resource utilization, and analyzing blockchain transaction logs for troubleshooting and performance analysis.

SOFTWARE REQUIREMENTS

- *Kali Linux / Ubuntu*
- *Docker*
- *Docker Compose*
- *Node.js (Version 18 or later)*
- *npm*
- *Git*
- *Hyperledger Fabric Samples*
- *Hyperledger Fabric Binaries*
- *Hyperledger Fabric Docker Images*
- *Visual Studio Code (Optional)*

HARDWARE REQUIREMENTS

- *Processor: Intel Core i3 or above*
- *RAM: Minimum 8 GB*
- *Storage: Minimum 20 GB Free Disk Space*
- *Internet Connection*

PROCEDURE

Step 1: Navigate to the Fabric Test Network cd

~/fabric-dev/fabric-samples/test-network

Step 2: Start the Hyperledger Fabric Network

Stop any existing Fabric network and start a fresh network with the application channel and Certificate Authorities.

NAME: VAISHNAVI E ROLL NO: 231901059

./network.sh down

./network.sh up createChannel -ca

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ cd ~/fabric-dev/fabric-samples/test-
network
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -ca
Using docker and docker compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' second
s and using database 'leveldb with crypto from 'Certificate Authorities'
Bringing up network
```

Step3Verify Running Container Display

all active Docker containers.

docker ps

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ docker ps
CONTAINER ID   IMAGE                               COMMAND                  CREATED         S
TATUS          PORTS                                                                       
                                                                        NAMES
2c5baef255f3   hyperledger/fabric-peer:latest      "peer node start"        31 seconds ago   U
p 30 seconds   0.0.0.0:9051->9051/tcp, [::]:9051->9051/tcp, 7051/tcp, 0.0.0.0:9445->9445/tcp,
[::]:9445->9445/tcp                                                     peer0.org2.example.com
a44b18b6714e   hyperledger/fabric-orderer:latest   "orderer"                31 seconds ago   U
p 30 seconds   0.0.0.0:7050->7050/tcp, [::]:7050->7050/tcp, 0.0.0.0:7053->7053/tcp, [::]:7053-
>7053/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp   orderer.example.com
ab1b97191d8a   hyperledger/fabric-peer:latest      "peer node start"        31 seconds ago   U
p 30 seconds   0.0.0.0:7051->7051/tcp, [::]:7051->7051/tcp, 0.0.0.0:9444->9444/tcp, [::]:9444-
>9444/tcp                                                               peer0.org1.example.com
14b030542e7e   hyperledger/fabric-ca:latest        "sh -c 'fabric-ca-se…"   45 seconds ago   U
p 44 seconds   0.0.0.0:7054->7054/tcp, [::]:7054->7054/tcp, 0.0.0.0:17054->17054/tcp, [::]:170
54->17054/tcp                                                           ca_org1
818d5fea0349   hyperledger/fabric-ca:latest        "sh -c 'fabric-ca-se…"   45 seconds ago   U
p 44 seconds   0.0.0.0:9054->9054/tcp, [::]:9054->9054/tcp, 7054/tcp, 0.0.0.0:19054->19054/tcp
, [::]:19054->19054/tcp                                                 ca_orderer
9d18e23bd759   hyperledger/fabric-ca:latest        "sh -c 'fabric-ca-se…"   45 seconds ago   U
p 44 seconds   0.0.0.0:8054->8054/tcp, [::]:8054->8054/tcp, 7054/tcp, 0.0.0.0:18054->18054/tcp
, [::]:18054->18054/tcp                                                 ca_org2
```

Step 4: Monitor CPU and Memory Usage

Display live CPU and memory utilization of the Fabric containers. docker stats

Allow the statistics to run for a few seconds and press Ctrl+C to stop. For a

single snapshot: docker stats --no-stream

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ docker stats --no-stream

CONTAINER ID   NAME                    CPU %     MEM USAGE / LIMIT     MEM %     NET I/O
      BLOCK I/O        PIDS
2c5baef255f3   peer0.org2.example.com   7.26%     26.55MiB / 5.625GiB   0.46%     225kB / 154k
B     45.1kB / 393kB   9
a44b18b6714e   orderer.example.com      0.39%     16.38MiB / 5.625GiB   0.28%     60.6kB / 207
kB    5.1MB / 193kB    8
ab1b97191d8a   peer0.org1.example.com   8.96%     30.62MiB / 5.625GiB   0.53%     225kB / 154k
B     0B / 393kB       8
14b030542e7e   ca_org1                  0.00%     9.527MiB / 5.625GiB   0.17%     40.1kB / 50k
B     0B / 737kB       7
818d5fea0349   ca_orderer               0.00%     11.05MiB / 5.625GiB   0.19%     60.5kB / 84.
6kB   999kB / 1.05MB   7
9d18e23bd759   ca_org2                  0.00%     9.633MiB / 5.625GiB   0.17%     37.9kB / 44k
B     1.16MB / 737kB   7
```

Step 5: View Peer Logs

Display the recent logs of the Org1

peer. docker logs --tail 50 peer0.org1.example.com

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ docker logs --tail 50 peer0.org1.exa
mple.com
2026-08-31 16:48:12.384 UTC 001c INFO [discovery] NewService -> Created with config TLS: true,
 authCacheMaxSize: 1000, authCachePurgeRatio: 0.750000
2026-08-31 16:48:12.384 UTC 001d INFO [nodeCmd] serve -> Discovery service activated
2026-08-31 16:48:12.384 UTC 001e INFO [nodeCmd] serve -> Starting peer with Gateway enabled
2026-08-31 16:48:12.384 UTC 001f INFO [nodeCmd] serve -> Starting peer with ID=[peer0.org1.exa
mple.com], network ID=[dev], address=[peer0.org1.example.com:7051]
2026-08-31 16:48:12.386 UTC 0020 INFO [nodeCmd] serve -> Started peer with ID=[peer0.org1.exam
ple.com], network ID=[dev], address=[peer0.org1.example.com:7051]
2026-08-31 16:48:12.387 UTC 0021 INFO [kvledger] LoadPreResetHeight -> Loading prereset height
 from path [/var/hyperledger/production/ledgersData/chains]
2026-08-31 16:48:12.387 UTC 0022 INFO [blkstorage] preResetHtFiles -> No active channels passe
d
2026-08-31 16:48:18.742 UTC 0023 INFO [ledgermgmt] CreateLedger -> Creating ledger [mychannel]
 with genesis block
2026-08-31 16:48:18.783 UTC 0024 INFO [blkstorage] newBlockfileMgr -> Getting block informatio
```

Step 6: View Orderer Logs

Display the recent logs of the ordering

service. docker logs --tail 50 orderer.example.com

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ docker logs --tail 50 orderer.exampl
e.com
 Version: v2.5.16
 Commit SHA: f871cf9
 Go version: go1.26.4
 OS/Arch: linux/amd64
2026-08-31 16:48:11.755 UTC 000f INFO [orderer.common.server] Main -> Beginning to serve reque
sts
2026-08-31 16:48:15.437 UTC 0010 INFO [blkstorage] newBlockfileMgr -> Getting block informatio
n from block storage
2026-08-31 16:48:15.466 UTC 0011 INFO [orderer.consensus.etcdraft] HandleChain -> EvictionSusp
icion not set, defaulting to 10m0s
2026-08-31 16:48:15.466 UTC 0012 INFO [orderer.consensus.etcdraft] HandleChain -> Without syst
em channel: after eviction Registrar.SwitchToFollower will be called
2026-08-31 16:48:15.467 UTC 0013 INFO [orderer.consensus.etcdraft] createOrReadWAL -> No WAL d
ata found, creating new WAL at path '/var/hyperledger/production/orderer/etcdraft/wal/mychanne
l' channel=mychannel node=1
```

Step 7: View Certificate Authority Logs

Display the logs of the Org1 Certificate

Authority. docker logs ca_org1

Step 8: Verify the Network Channel

Load the Org1 peer environment and list the channels joined by the peer. source

setOrg1.sh peer channel list

Step 9: Deploy the Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer chaincode to mychannel.

./network.sh deployCC -ccl javascript -ccn basic -ccp ../assettransferbasic/chaincode-javascript

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer channel list
2026-08-31 16:51:48.475 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer conn
ections initialized
Channels peers has joined:
mychannel
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ source setOrg1.sh
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl javascrip
t -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
Using docker and docker compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
```

NAME: VAISHNAVI E ROLL NO: 231901059

Step 10: Query Installed Chaincode

Verify that the chaincode package has been installed on the peer. peer

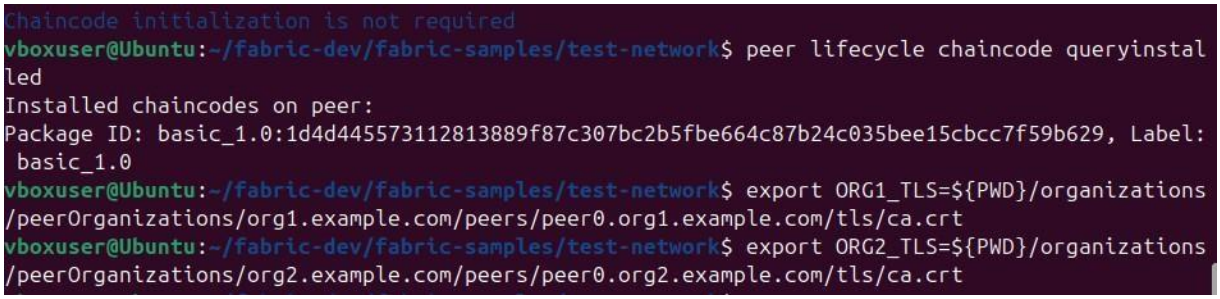
lifecycle chaincode queryinstalled

Step 11: Set TLS Environment Variables

Set the TLS certificates for Org1 and

Org2. export ORG1_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peer s /peer0.org1. example.com/tls/ca.crt

export ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peer s /peer0.org2. example.com/tls/ca.crt



Step 12: Initialize the Ledger

Initialize the ledger with the sample assets. Both Org1 and Org2 peers are included for endorsement.

peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function":"InitLedger","Args":[]}' **Step 13: Monitor Blockchain Transactions** *Query all assets*

stored in the ledger. peer chaincode query -C

mychannel -n basic -c '{"Args":["GetAllAssets"]}'

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7
050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -
n basic --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses localhost:
9051 --tlsRootCertFiles $ORG2_TLS -c '{"function":"InitLedger","Args":[]}'
2026-08-31 16:53:30.660 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invok
e successful. result: status:200
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n
 basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset
"},{"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"
},{"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"a
sset"},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":
"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docT
ype":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"
docType":"asset"}]
```

Step 14: View Recent Docker Events

Monitor Docker events generated by the Fabric network during the previous ten minutes.

docker events --since 10m Press Ctrl+C after capturing the required events. For a fixed monitoring period: timeout 15 docker events -since 10m

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ timeout 15 docker events --since 10m
2026-08-31T16:47:52.574800210Z volume create d07763cff4f20fcee772a4c97f51188990dfa41589560306b
47535e5abecf1bc (driver=local)
2026-08-31T16:47:52.578867986Z volume create aef332f7adc6ef0b0db7cf7d7899ea9a94a7403a9de15b011
e28935df1a590f9 (driver=local)
2026-08-31T16:47:52.619590321Z container create 5351fdef77e8a42e226d7c9e7ccb30d45c1db165e69f6b
423c02cc9002557038 (image=hyperledger/fabric-peer:latest, name=hardcore_torvalds, org.opencont
ainers.image.created=2026-06-17T22:30:08.458Z, org.opencontainers.image.description=Hyperledge
r Fabric is an enterprise-grade permissioned distributed ledger framework for developing solut
ions and applications. Its modular and versatile design satisfies a broad range of industry us
e cases. It offers a unique approach to consensus that enables performance at scale while pres
erving privacy., org.opencontainers.image.licenses=Apache-2.0, org.opencontainers.image.revisi
on=f871cf92a026aba7b12e6f06d71ded3e6e659d71, org.opencontainers.image.source=https://github.co
m/hyperledger/fabric, org.opencontainers.image.title=fabric, org.opencontainers.image.url=http
s://github.com/hyperledger/fabric, org.opencontainers.image.version=v2.5.16)
```

Step 15: Stop the Fabric Network

Stop and remove the Fabric network.

./network.sh down

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker compose
Stopping network
[+] down 1/6
 ⠿ Container ca_org1                Stopping                                    1.3s
 ⠿ Container ca_org2                Stopping                                    1.3s
 ⠿ Container peer0.org1.example.com Stopping                                    1.3s
 ✔ Container orderer.example.com    Removed                                     1.1s
 ⠿ Container peer0.org2.example.com Stopping                                    1.3s
 ⠿ Container ca_orderer             Stopping                                    1.3s
```

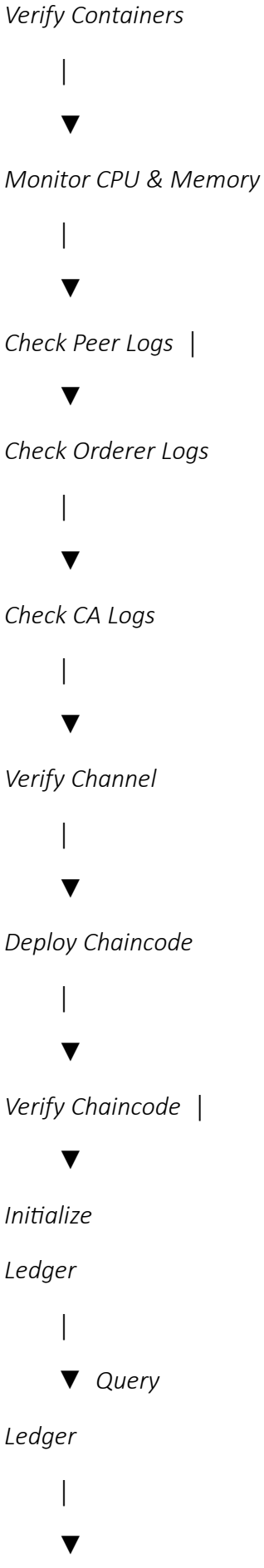
MONITORING WORKFLOW

Start Fabric Network

|

▼

NAME: VAISHNAVI E ROLL NO: 231901059



NAME: VAISHNAVI E ROLL NO: 231901059

Monitor Docker Events

|

Stop Fabric Network

RESULT

The Hyperledger Fabric network was successfully monitored using Docker utilities and Fabric CLI commands. Peer nodes, the ordering service, Certificate Authorities, channels, chaincode installation, and blockchain ledger data were verified through monitoring commands and log analysis. The experiment demonstrated effective monitoring and troubleshooting techniques for maintaining a reliable Hyperledger Fabric network.